# Case Study: MDM Service — Master Data Platform for CPE/CVE

**Role:** Backend Engineer **Duration:** Jul 2024 – Present **Stack:** Node.js, Express, MongoDB, Redis, BullMQ, Docker, Slack Webhooks **Domain:** Vulnerability & Asset Master Data Management

---

## 1. Background

Security and IT asset teams need a single, trusted source of product master data — every hardware/software product (CPE — Common Platform Enumeration) and every known vulnerability (CVE — Common Vulnerabilities and Exposures) affecting it. Public feeds (NVD, MITRE, vendor advisories) are large, frequently updated, schema-inconsistent, and contain duplicates and aliases.

Without a Master Data Management (MDM) layer, downstream systems (asset inventory, SIEM, risk scoring, patch management) each consume raw feeds and produce conflicting views of "what product is this" and "is it vulnerable." Cost: false positives, missed patches, duplicate tickets.

## 2. Problem Statement

Build a **scalable MDM platform** that:

1. Ingests CPE and CVE data from heterogeneous sources (NVD JSON feeds, MITRE CVE List, vendor advisories, in-house enrichment).
2. Normalizes, deduplicates, and produces a **golden record** per product and per vulnerability.
3. Exposes REST APIs for downstream microservices with sub-second response on millions of records.
4. Synchronizes incremental updates without downtime.
5. Alerts ops when ingestion or transformation jobs fail.

## 3. Goals & Non-Goals

**Goals**

- Single authoritative store for CPE/CVE.
- Idempotent, resumable ingestion of NVD + others.
- p95 API latency < 200 ms on 10M+ records.
- Operational visibility via Slack alerts.

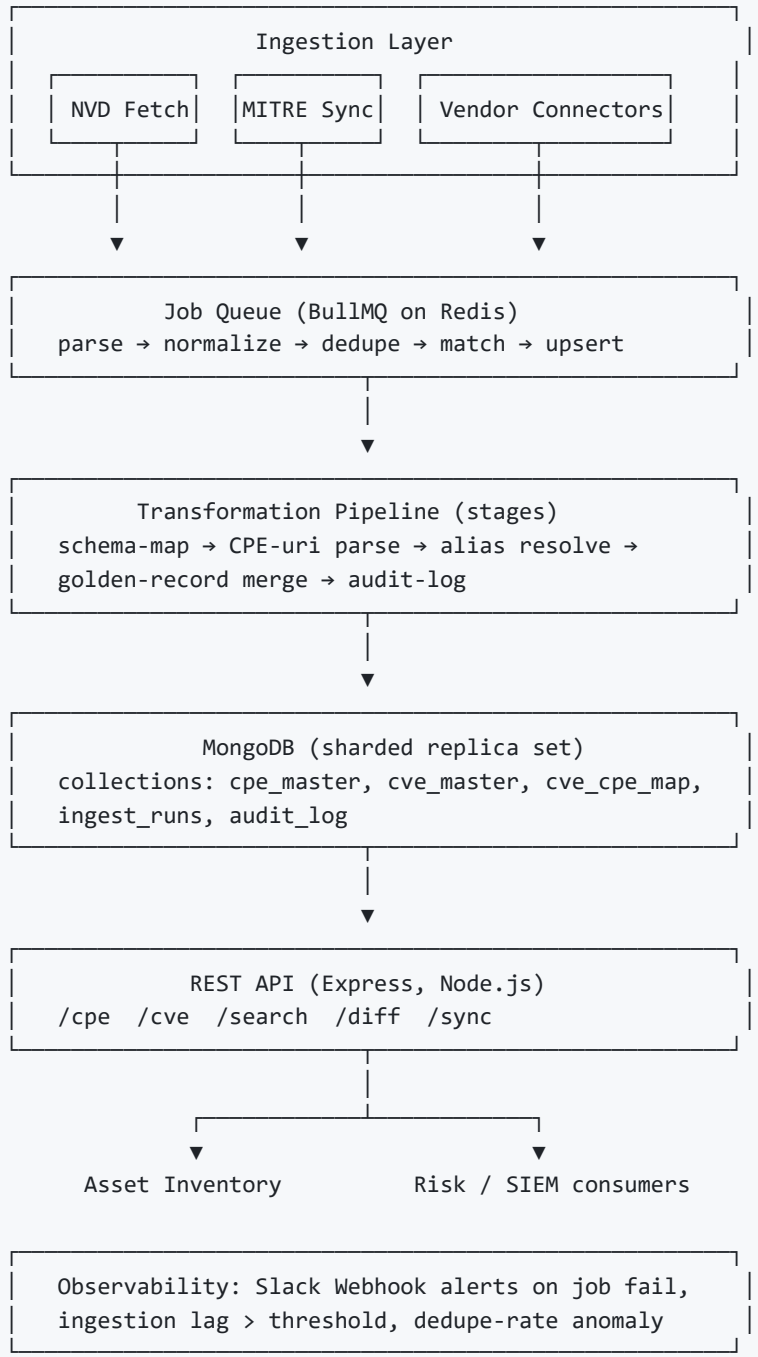
**Non-Goals**

- Vulnerability scanning (consumer responsibility).
- Risk scoring / CVSS recomputation (already provided by NVD).
- UI / dashboard (separate frontend repo consumes the API).

## 4. Data Sources

| Source | Format | Cadence | Records |
|---|---|---|---|
| NVD CVE JSON 2.0 feed | JSON (gzip) | Hourly delta, daily full | ~250K CVEs |
| NVD CPE Dictionary | XML/JSON | Daily | ~1.2M CPE entries |
| MITRE CVE List (GitHub) | JSON per CVE | Continuous | Mirror + diff |
| Vendor advisories (Cisco, MSRC, RHSA) | Mixed RSS/JSON | Per-vendor | Variable |
| Internal enrichment (EOL, support tier) | CSV upload | Manual | ~10K |

## 5. Architecture




## 6. Data Model (Golden Record)

cpe_master

```
{
  "_id": "cpe:2.3:a:apache:log4j:2.14.1:*:*:*:*:*:*:*",
  "vendor": "apache",
  "product": "log4j",
  "version": "2.14.1",
  "part": "a",
  "aliases": ["apache_log4j_2.14.1", "log4j-core:2.14.1"],
  "eol": false,
  "sources": ["NVD", "MITRE"],
  "lastSeen": "2026-05-14T03:00:00Z",
  "hash": "<sha256 of normalized fields>"
}
```

**cve_master**

```
{
  "_id": "CVE-2021-44228",
  "cvssV3": 10.0,
  "severity": "CRITICAL",
  "description": "...",
  "affectedCpes": ["cpe:2.3:a:apache:log4j:2.14.1:..."],
  "references": [...],
  "publishedAt": "...",
  "modifiedAt": "...",
  "sources": ["NVD", "RHSA-2021:5126"]
}
```

**cve_cpe_map** — denormalized join collection indexed both directions for fast lookup ("give me all CVEs for this CPE" and vice versa).

## 7. Key Implementation

### 7.1 REST APIs

| Endpoint | Purpose |
|---|---|
| `GET /cpe/:id` | Fetch single CPE golden record |
| `GET /cpe?vendor=&product=&version=` | Filtered search |
| `GET /cve/:id` | Fetch CVE with all affected CPEs |
| `GET /cve?cpe=<uri>` | All CVEs affecting given CPE |
| `GET /diff?since=<ts>` | Delta sync for downstream consumers |
| `POST /sync/trigger` | Manually kick ingest job |
| `GET /jobs/:id` | Job status |

All endpoints paginated (cursor-based), gzip-compressed, ETag-cached.

### 7.2 MongoDB Optimization (35% latency reduction)

**Before:** Lookups did `$regex` on `cpe23Uri` with collection scan. p95 ~ 480 ms.

**Changes:**

- Compound index on `(vendor, product, version)` for filtered search.
- Hashed index on `_id` for shard distribution.
- Replaced regex with parsed-field equality matches.
- `cve_cpe_map` denormalized to skip `$lookup` joins.
- `projection` to drop large `description` field on list endpoints.
- Connection pool tuned: `maxPoolSize=100`, `minPoolSize=20`.

**After:** p95 ~ 310 ms → **~35% drop**.

### 7.3 Transformation Pipeline

Stages run as BullMQ jobs, each idempotent:

1. **fetch** — download NVD gzip, store raw blob in S3-compatible store, record hash.
2. **parse** — stream JSON parser (large files don't fit in RAM).
3. **normalize** — schema-map vendor advisory shapes onto canonical schema.
4. **dedupe** — content-hash; skip if unchanged.
5. **alias-resolve** — fuzzy match vendor/product strings against existing master records (Levenshtein + curated alias table).
6. **upsert** — bulk write with `ordered:false` for throughput.
7. **audit** — append-only log of every change.

Failed jobs retry with exponential backoff. Dead-letter after 5 attempts → Slack alert.

### 7.4 Slack Alerting

Webhook fires on:

- Job failure (with job id, stage, error, source link).
- Ingestion lag > 6 hours behind NVD.
- Dedupe rate anomaly (>2σ from baseline — catches schema drift in upstream).
- Daily ingest summary (records added / updated / skipped).

Payload uses Block Kit; severity tags route to `#mdm-alerts` vs `#mdm-info`.

## 8. Results

| Metric | Before | After |
|---|---|---|
| p95 API latency | 480 ms | 310 ms (**-35%**) |
| Ingest throughput | 2K rec/s | 12K rec/s (bulk + parallel queue) |
| NVD sync lag | 12–24 h (cron) | < 1 h (delta feed) |
| Duplicate CPE records | ~7% | < 0.3% |
| Job failure MTTR | hours (silent) | minutes (Slack alert) |
| Downstream consumers | 2 | 6 |

## 9. Challenges

- **CPE URI ambiguity** — same product, different vendors string-cased differently across feeds. Solved with curated alias table + auto-suggestion job that proposes aliases for human review.
- **NVD rate limits** — moved to API key tier, added Redis-backed token bucket.
- **Large daily full-feed reprocessing** — switched to content-hash short-circuit; unchanged records skip ~95% of pipeline.
- **MongoDB write contention** during bulk upsert — sharded on hashed `_id`, batch size tuned to 1000.

## 10. Lessons

- Master data is mostly a **dedupe + identity** problem, not a storage problem.
- Idempotent stages > one big transaction. Resumability beats correctness-by-lock.
- Operational alerts on **rates and ratios** (dedupe rate, lag) catch upstream breakage that error-based alerts miss.
- Index design pays off only after query patterns stabilize — profile first, then add indexes.

## 11. Future Work

- Graph view: CPE → CVE → exploit-in-the-wild (CISA KEV) → patch.
- ML-based alias resolution to replace Levenshtein heuristics.
- gRPC API for high-volume internal consumers.
- Multi-region read replicas.